Laboratório Funcional · Aula 12

BRUNO VINICIUS COUTINHO MIANTE DA SILVA · RA 24151605-2 · Turma ESOFT C
Gerado em 20/10/2026, 19:45:25 · tempo registrado no cronômetro: 0:00

Preparação: Programar com funções matemáticas

def area(r): return 3.14159 * r * r

Pura

def agora(): return datetime.now()

Impura

def total(lista): return sum(lista)

Pura

def registra(msg): print(msg)

Impura

def sorteia(): return random.randint(1, 6)

Impura

def poe(lista, x): lista.append(x); return lista

Impura

0.1 Escolha duas funções que você marcou como impuras e explique, para cada uma, o que a impede de ser pura.

A função agora() é impura porque depende do relógio do sistema e pode devolver resultados diferentes sem parâmetros. registra(msg) é impura porque imprime na tela, causando efeito colateral. Também sorteia() depende de um gerador aleatório e poe() modifica a lista recebida.

Prática 1: Funções puras e formas funcionais

Previsão: O que as linhas 1, 4 e 7 vão imprimir?

1: A-1 A-2. 4: 14. 7: [7, 3, 9] [3, 7, 9].

Saídas (original e com compor(g, f))

```
Saída original:
1: A-1 A-2
2: True
3: 8
4: 14
5: [4, 9, 16]
6: 29
7: [7, 3, 9] [3, 7, 9]
8: [3, 7, 9]

Com compor(g, f): linha 4 = 18.
```

1.1 Por que proximo_id("A") devolve valores diferentes para a mesma entrada? Por que isso dificulta os testes (p. 637)?

proximo_id altera o contador global a cada chamada, por isso a mesma entrada A produz A-1, depois A-2. Isso dificulta os testes porque o resultado depende do estado anterior e a ordem de execução dos testes.

1.2 compor(f, g)(4) e compor(g, f)(4) deram valores diferentes. Calcule os dois à mão e explique a diferença.

compor(f,g)(4) = f(g(4)): g(4)=12 e f(12)=14. Já compor(g,f)(4)=g(f(4)): f(4)=6 e g(6)=18. A ordem de composição muda o resultado.

1.3 Qual é a diferença entre sorted(notas) e notas.sort()? Qual dos dois segue o estilo funcional, e por quê?

sorted(notas) cria e devolve uma nova lista ordenada sem alterar notas. notas.sort() modifica a própria lista. sorted segue melhor o estilo funcional porque evita modificar o dado original.

Prática 2: Scheme: tudo é lista

Previsão: (car '(A B C))

A

Previsão: (cdr '(A B C))

(B C)

Previsão: (cons '(A B) '(C D))

((A B) C D)

Saídas (original e sem o apóstrofo)

```
20
14
25
8
positivo
B
(C)
(A B C)
((A B) C D)
B

Sem apóstrofo, (car (A B C)) causa erro porque Scheme tenta avaliar A como uma
expressão/função.
```

2.1 Para que serve o apóstrofo (')? O que o Scheme tentou fazer com (A B C) quando ele não estava lá?

O apóstrofo impede a avaliação da lista e faz com que ela seja tratada como dado literal. Sem ele, Scheme tenta executar (A B C) como uma chamada de função, mas A não está definido como procedimento.

2.2 Escreva uma expressão que devolva o terceiro elemento de '(A B C) usando só car e cdr. Teste no OneCompiler e cole aqui a expressão e o resultado.

```
(car (cdr (cdr '(A B C)))) ⇒ C
```

Prática 3: Recursão e recursão em cauda

O que vi no passo a passo

não respondido

Previsão: O programa em Scheme chama conta-ate recursivamente um milhão de vezes. Vai funcionar? E a mesma função em Python?

No Scheme, a versão em cauda deve conseguir contar até um milhão se houver otimização de chamada em cauda. Em Python, deve ocorrer RecursionError porque o interpretador limita a profundidade da recursão.

Saída do Scheme

```
#t
#f
2432902008176640000
2432902008176640000
1000000
```

Saída do Python (as últimas linhas bastam)

```
500
RecursionError: maximum recursion depth exceeded
```

3.1 No passo a passo, o que cresce na recursão comum e não cresce na recursão em cauda? Por quê?

Na recursão comum cresce a pilha de chamadas, pois cada chamada precisa esperar o resultado da próxima. Na recursão em cauda, não há operação pendente após a chamada recursiva, então uma implementação com otimização pode reutilizar o quadro atual.

3.2 Por que o Scheme fez um milhão de chamadas e o Python parou com RecursionError? (Dica: releia o quadro sobre a otimização da chamada em cauda e a mensagem de erro do Python. O livro fala do Scheme na p. 656.)

Scheme pode otimizar chamadas em cauda e executar a repetição sem acumular um milhão de quadros de pilha. Python não implementa essa otimização e tem limite de recursão, por isso a chamada termina em RecursionError.

Prática 4: Mão na massa: três funções sobre listas

Saída final (com os três ok)

```
tamanho: 3  ok
soma-quadrados: 14  ok
pares: (2 4 6)  ok
```

4.1 Cole aqui as suas três funções (tamanho, soma-quadrados e pares).

```
(define (tamanho lis)
  (if (null? lis) 0
      (+ 1 (tamanho (cdr lis)))))

(define (soma-quadrados lis)
  (if (null? lis) 0
      (+ (* (car lis) (car lis))
         (soma-quadrados (cdr lis)))))

(define (pares lis)
  (cond ((null? lis) '())
        ((even? (car lis)) (cons (car lis) (pares (cdr lis))))
        (else (pares (cdr lis)))))
```

4.2 Qual foi a parte mais difícil? Onde você usou null?, car e cdr em cada função?

O mais difícil foi organizar os casos recursivos. Em todas as funções, null? verifica a lista vazia e cdr avança para o restante. car acessa o primeiro elemento; em pares, even? decide se ele entra no resultado e cons constrói a nova lista.

Prática 5: Haskell: padrões, compreensões e preguiça

Previsão: squares é uma lista infinita. O que você acha que take 5 squares imprime? E primeiro 42 (div 1 0)?

take 5 squares deve imprimir [0,1,4,9,16]. primeiro 42 (div 1 0) deve devolver 42, pois Haskell avalia o segundo argumento apenas se for necessário.

Saídas (original e com member 17)

```
2432902008176640000
[1,2,4,7,14]
[2,2,4,7,8,9]
[0,1,4,9,16]
[0,1,4,9,16,25,36,49]
True
42

Com member 17 squares, a busca não termina, pois 17 não aparece na lista de quadrados.
```

5.1 Como o Haskell consegue imprimir parte de uma lista infinita? E por que primeiro 42 (div 1 0) não deu erro de divisão por zero?

Haskell usa avaliação preguiçosa: calcula apenas os elementos solicitados, então take pode obter uma parte finita de uma lista infinita. primeiro retorna seu primeiro argumento sem avaliar o segundo, por isso a divisão por zero não é executada.

5.2 O que aconteceu com member 17 squares? Por que ele nunca termina, enquanto member 16 squares terminou? Como você corrigiria? (Dica: squares está em ordem crescente. Quando já dá para desistir da busca? O livro mostra uma solução na p. 670.)

member 17 squares procura indefinidamente porque 17 não é um quadrado e a lista cresce sem fim. member 16 termina porque encontra 16. Como squares está em ordem crescente, a busca pode parar quando o elemento atual for maior que o valor procurado.

Prática 6: Você já usa funcional

Previsão: O que mais10(5) e [1, 2, 3].map(vezes(5)) vão imprimir?

mais10(5) imprime 15. [1, 2, 3].map(vezes(5)) imprime [5, 10, 15].

Saída do JavaScript

```
15
[5, 10, 15]
56
{ nome: 'Ana', nota: 7 } { nome: 'Ana', nota: 8 }
```

Saída do Java

```
14
18
```

```
1050.0
[CABO, MONITOR, MOUSE, TECLADO]
```

6.1 somador(10) já terminou de executar quando mais10(5) é chamado. Como a função devolvida ainda "lembra" o 10?

A função interna criada por somador(10) fecha sobre o ambiente léxico em que foi definida. Esse closure mantém uma referência ao valor n=10 mesmo depois que somador termina.

6.2 Por que f.compose(g).apply(4) e f.andThen(g).apply(4) deram resultados diferentes? Qual deles corresponde ao compor(f, g) da Prática 1?

compose aplica primeiro a função recebida e depois a função sobre a qual o método foi chamado: f.compose(g)(4)=f(g(4))=14. andThen faz f primeiro e g depois: g(f(4))=18. compose corresponde a compor(f,g).

Prática 7: O laço vira pipeline

7.1 Primeiro exercício escolhido (1, 2 ou 3): cole o seu código reescrito.

```
Exercício 1 (Java):
int soma = nums.stream().filter(n → n % 2 == 0).map(n → n *
n).mapToInt(Integer::intValue).sum();
System.out.println(soma);
```

7.2 Segundo exercício escolhido: cole o seu código reescrito.

```
Exercício 2 (JavaScript):
const aprovados = alunos.filter(a ⇒ a.nota ⩾ 6).map(a ⇒ a.nome.toUpperCase());
console.log(aprovados.join(", "));
```

Saídas das suas versões

```
Exercício 1: 244
Exercício 2: ANA, CARLA, DAVI
```

7.3 Qual versão é mais fácil de ler: a com laço ou a funcional? E qual é mais fácil de depurar (por exemplo, para ver um valor intermediário)?

A versão funcional costuma ser mais curta e deixa explícitas as etapas de filtrar e transformar os dados. A versão com laço pode ser mais fácil de depurar passo a passo, pois permite inspecionar cada variável intermediária.

Fechamento: O que fica do paradigma funcional

Uma função pura pode ler uma variável global que muda.

Falso

Em Scheme, a recursão em cauda não faz a pilha crescer.

Verdadeiro

O Python otimiza a recursão em cauda.

Falso

Em Haskell, [1 ..] só é calculada à medida que é usada.

Verdadeiro

filter, map e reduce dispensam laço e variável acumuladora.

Verdadeiro

Currying transforma uma função de vários parâmetros numa cadeia de funções de um parâmetro.

Verdadeiro

8.1 Qual versão é mais fácil de entender, e por quê? O livro estima que um programa funcional costuma levar cerca de 2 vezes mais tempo para executar (p. 678). Em que tipo de sistema você aceitaria esse custo, e em qual não?

A versão funcional com map e sum é concisa e mostra a transformação dos dados; a versão com laço pode ser mais intuitiva para quem está começando. Aceitaria maior tempo de execução em sistemas de análise, relatórios ou processamento em lote, priorizando clareza e manutenção. Evitaria esse custo em sistemas de tempo real ou aplicações com limites rígidos de desempenho.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que mais me surpreendeu foi que funções podem guardar contexto e que Haskell consegue trabalhar com listas infinitas calculando apenas o necessário.

8.3 Qual é o erro de impressão, e como corrigir? O que a mensagem do compilador diz sobre o tipo que ele inferiu?

Na compreensão de lista, a expressão b <- h está errada porque h é um valor, não uma lista geradora. O correto é b > h. O compilador indica incompatibilidade de tipos: esperava uma lista/estrutura iterável, mas encontrou um valor do tipo do elemento.